

Módulo 4: Fundamentos de Machine Learning

Introducción

Machine Learning permite construir modelos que aprenden patrones a partir de ejemplos. El modelo no reemplaza el razonamiento: depende de los datos, las variables elegidas, la evaluación y la interpretación de sus límites.

En este módulo seguirás un flujo mínimo y reproducible con Scikit-Learn:

`datos -> variables y objetivo -> entrenamiento/prueba -> modelo -> predicción -> evaluación`

Reto del módulo

Pregunta guía: ¿Podemos clasificar una medición ambiental como `revisar` o `normal` usando temperatura y humedad?

Producto final: un clasificador reproducible con evaluación, matriz de confusión y una conclusión prudente sobre su desempeño.

Criterios de éxito:

- distingues variables predictoras y objetivo;
- separas datos de entrenamiento y prueba;
- entrenas un modelo sin usar el objetivo como predictor;
- evalúas predicciones con más de una métrica;
- interpretas errores y limitaciones;
- evitas presentar un modelo pequeño como una verdad general.

1. Conceptos esenciales

Concepto	Significado
observación	una fila del dataset
característica (X)	variable usada para predecir
objetivo (y)	respuesta que se quiere predecir
entrenamiento	datos usados para ajustar el modelo
prueba	datos reservados para evaluar
predicción	salida producida por el modelo
métrica	medida del desempeño

En este reto:

- X: temperatura y humedad;
- y: etiqueta revisar o normal;
- cada fila: una medición ambiental.

2. Preparar los datos

```
import pandas as pd

X = datos[["temperatura", "humedad"]]
y = datos["estado"]
```

Revisa primero:

```
datos.shape
datos.dtypes
datos["estado"].value_counts()
datos.isna().sum()
```

Preguntas de control:

1. ¿Hay suficientes ejemplos de cada clase?
2. ¿Hay valores faltantes?
3. ¿Las unidades son consistentes?
4. ¿La variable objetivo aparece accidentalmente dentro de X?

3. Separar entrenamiento y prueba

```
from sklearn.model_selection import train_test_split

X_entrenamiento, X_prueba, y_entrenamiento, y_prueba = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y,
)
```

El conjunto de prueba debe permanecer separado hasta la evaluación final. `random_state` permite repetir el experimento.

`stratify=y` procura conservar una proporción parecida de clases en ambos conjuntos. Esto es importante cuando una clase es menos frecuente.

4. Entrenar un modelo

Un modelo de árbol toma decisiones mediante condiciones sobre las características.

```
from sklearn.tree import DecisionTreeClassifier

modelo = DecisionTreeClassifier(max_depth=3, random_state=42)
modelo.fit(X_entrenamiento, y_entrenamiento)

predicciones = modelo.predict(X_prueba)
```

`fit` aprende a partir de ejemplos. `predict` genera respuestas para datos que el modelo no recibió durante el entrenamiento.

Error frecuente: fuga de información

No uses columnas que revelen directamente el objetivo ni calcules transformaciones usando todo el dataset antes de separar los datos. La prueba debe simular datos futuros o no vistos.

5. Evaluar el desempeño

Exactitud

```
from sklearn.metrics import accuracy_score

exactitud = accuracy_score(y_prueba, predicciones)
```

La exactitud es la proporción de predicciones correctas. Puede ser engañosa si una clase domina el dataset.

Matriz de confusión

```
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix

matriz = confusion_matrix(y_prueba, predicciones, labels=["normal", "revisar"])
ConfusionMatrixDisplay(
    confusion_matrix=matriz,
    display_labels=["normal", "revisar"],
).plot()
```

La matriz permite contar:

- verdaderos normales;
- verdaderos casos para revisar;
- falsos positivos;
- falsos negativos.

En un sistema de alerta, un falso negativo puede ser más costoso que un falso positivo. La métrica adecuada depende del contexto.

Reporte de clasificación

```
from sklearn.metrics import classification_report

print(classification_report(y_prueba, predicciones, zero_division=0))
```

- precisión: de los casos marcados como una clase, cuántos eran correctos;
- recall: de los casos reales de una clase, cuántos encontró el modelo;
- F1: equilibrio entre precisión y recall.

6. Interpretar sin exagerar

Un resultado alto en un dataset pequeño no garantiza que el modelo funcione en otro lugar o momento.

Incluye siempre:

- tamaño del dataset;
- variables usadas;
- forma de separación;
- métricas y matriz de confusión;
- posibles sesgos o datos faltantes;
- siguiente dato que sería necesario recolectar.

Una conclusión responsable se parece a esto:

> En este conjunto de prueba, el modelo clasificó correctamente la mayoría de las mediciones. El resultado es exploratorio porque hay pocos ejemplos y solo usamos dos variables; se necesita validar con datos nuevos antes de automatizar alertas.

7. Actividad del estudiante

Abre `01_flujo_machine_learning.ipynb` y completa `02_reto_modelo.py`.

Reto 1: reproducción

Prepara `X` e `y`, separa los datos y entrena un árbol de decisión.

Reto 2: evaluación

Calcula exactitud y matriz de confusión. Identifica qué tipo de error aparece más veces.

Reto 3: extensión

Cambia `max_depth`, compara el resultado y explica si el cambio puede aumentar el sobreajuste.

Evidencia y evaluación

Criterio	Logrado cuando...
Preparación	separas características y objetivo correctamente
Reproducibilidad	usas una semilla y documentas el flujo
Evaluación	reportas exactitud y matriz de confusión
Interpretación	distingues desempeño de generalización
Ética	nombras límites, riesgos y datos faltantes
Comunicación	explicas el resultado con lenguaje no absoluto

Reflexión final

1. ¿Qué diferencia hay entre entrenar y evaluar un modelo?
2. ¿Cuándo puede ser peligrosa una exactitud alta?
3. ¿Qué dato adicional usarías para decidir si una medición requiere revisión?